SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

ASSIGNMENT 3 - C++ PROGRAMMING

NAME:DENISHA SHREE R

REG NO:192524016

Q1 – Constructors and Constructor Overloading

Problem: A library creates Book objects as a full book, a donated book with only a title, or an empty record. The class must use a default constructor, parameterised/overloaded constructors, member initializer lists and constructor delegation, while rejecting invalid negative copy counts.

Code

```
#include <iostream>
#include <string>
class Book
{
private:
    std::string title;
    std::string author;
    int copies;
    static int validCopies(int c)
    {
        return (c < 0) ? 0 : c;
    }
public:
    // Default constructor
    Book()
        : Book("", "", 0)
    {
    }
    // Title-only constructor
    explicit Book(const std::string& t)
        : Book(t, "", 0)
    {
    }
    // Full constructor
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(validCopies(c))
    {
    }
```

```
    void print() const
    {
        std::cout << "Title: " << title
                  << ", Author: " << author
                  << ", Copies: " << copies << '\n';
    }
};
int main()
{
    Book b1;
    Book b2("C++ Programming");
    Book b3("Data Structures", "Bjarne", 5);
    Book b4("Invalid Book", "Author", -3);
    b1.print();
    b2.print();
    b3.print();
    b4.print();
    return 0;
}
```

Expected output:



Design and Analysis

- The compiler supplies a default constructor only when the class does not declare any constructor. Once a user-declared constructor exists, the compiler does not automatically provide the no-argument default constructor.
- Member initializer lists are preferred because members are initialized directly instead of being default-constructed and then assigned. They are required for const members, reference members and base classes.
- Members are initialized in the order of their declaration inside the class, not the order written in the initializer list. Writing the list in another order can produce a -Wreorder warning.
- Constructor delegation lets one constructor call another constructor of the same class. This keeps validation of the copy count in one place.
- Time for constructing one Book is O(1) apart from the string work; the fixed set of members uses O(1) object storage.

State of the Three Main Objects

b1 -> title="", author="", copies=0

b2 -> title="C++ Programming", author="", copies=0

b3 -> title="Data Structures", author="Bjarne", copies=5

Explicit Constructor

The title-only constructor is marked explicit so that a string cannot silently become a Book. Without explicit, Book b = "C++ Programming" would compile. With explicit, the direct form Book b("C++ Programming") is allowed, but the implicit copy-initialization Book b = "C++ Programming" is rejected.

Alternative Design

One constructor with default arguments, such as Book(std::string t="", std::string a="", int c=0), can replace several overloads. It is shorter, but overloads and delegation make the three intended construction cases clearer. Both designs require O(1) object-creation work for the fixed fields. The delegated design performs the copy-count validation in one place.

Q2 – Copy Constructors: Shallow and Deep Copies

A Prescription owns a dynamically allocated array of medicine names. A shallow copy would copy only the pointer, so two objects would refer to the same array. A deep copy allocates a separate array and copies each medicine.

Code

```
#include <iostream>
#include <string>
class Prescription
{
   int n;
   std::string* medicines;
public:
   explicit Prescription(int count)
      : n(count), medicines(new std::string[count])
   {
   }
   // TODO 1: Deep-copy constructor
   Prescription(const Prescription& other)
      : n(other.n), medicines(new std::string[other.n])
   {
      for (int i = 0; i < n; ++i)
         medicines[i] = other.medicines[i];
   }
   // TODO 2: Deep-copy assignment
   Prescription& operator=(const Prescription& other)
   {
      if (this != &other)
      {
         std::string* newMedicines =
            new std::string[other.n];
         for (int i = 0; i < other.n; ++i)
            newMedicines[i] = other.medicines[i];
         delete[] medicines;
         medicines = newMedicines;
         n = other.n;
      }
      return *this;
   }
   // TODO 3: Destructor
   ~Prescription()
   {
      delete[] medicines;
   }
   void set(int i, const std::string& m)
   {
      medicines[i] = m;
```

```
    }
    std::string get(int i) const
    {
        return medicines[i];
    }
};
int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << " " << p.get(1) << " | "
              << q.get(0) << " " << q.get(1) << " | "
              << r.get(0) << " " << r.get(1) << '\n';
    // Self-assignment test
    p = p;
    return 0;
}
```

Expected output:



Memory Diagrams

SHALLOW COPY

p.medicines ----+

|

q.medicines ----+----> [Paracetamol][ORS]

Both objects own the same array.

Two destructors would call delete[] on the same address.

DEEP COPY

p.medicines ----------> [Paracetamol][ORS]

q.medicines ----------> [Paracetamol][ORS]

Each object owns a different array.

Changing q does not change p.

Rule of Three

If a class manually manages a resource, it normally needs a destructor, a copy constructor and a copy-assignment operator. These three functions form the Rule of Three because copying or destroying the resource must be handled consistently.

When Is the Copy Constructor Called?

- When an object is initialized from another object, e.g. Prescription q = p.
- When an object is passed by value to a function.
- When an object is returned by value when copy elision does not remove the copy.

Passing by const reference does not copy the object. A shallow copy is O(1) to copy the pointer, but it is incorrect for ownership. A deep copy takes O(n) time and O(n) additional memory for n medicines.

Rule of Zero Alternative

Using std::vector<std::string> instead of a raw dynamically allocated array allows the compiler-generated copy operations to perform the correct deep copy automatically. This is the Rule of Zero approach and is generally safer.

Q3 – Dynamic Constructors and Destructors

The auditorium stores a dynamically allocated two-dimensional seat map. The constructor allocates the rows and each row, while the destructor releases each row and then the array of row pointers.

Code

```
#include <iostream>
#include <string>
class Event
{
   int rows;
   int seatsPerRow;
   char** seats;
   std::string name;
public:
   Event(const std::string& eventName, int r, int s)
      : rows(r), seatsPerRow(s), seats(nullptr), name(eventName)
   {
      if (rows <= 0 || seatsPerRow <= 0)
      {
         rows = 0;
         seatsPerRow = 0;
         std::cout << "Event created with empty seat map\n";
         return;
      }
      seats = new char*[rows];
      for (int i = 0; i < rows; ++i)
      {
         seats[i] = new char[seatsPerRow];
         for (int j = 0; j < seatsPerRow; ++j)
            seats[i][j] = 'O';
      }
      std::cout << "Event created: " << name << '\n';
   }
   ~Event()
   {
      if (seats != nullptr)
      {
         for (int i = 0; i < rows; ++i)
            delete[] seats[i];
         delete[] seats;
      }
      std::cout << "Event destroyed: " << name
            << " (" << rows << " x "
            << seatsPerRow << ")\n";
   }
   void book(int r, int s)
```

```
    {
        if (r >= 0 && r < rows &&
            s >= 0 && s < seatsPerRow)
        {
            seats[r][s] = 'X';
        }
    }
    void printMap() const
    {
        for (int i = 0; i < rows; ++i)
        {
            for (int j = 0; j < seatsPerRow; ++j)
                std::cout << seats[i][j] << ' ';
            std::cout << '\n';
        }
    }
};
int main()
{
    Event e1("Stack Event", 2, 4);
    {
        Event e2("Block Event", 3, 3);
        e2.book(1, 1);
        e2.printMap();
    }
    Event* e3 = new Event("Heap Event", 2, 3);
    e3->book(0, 2);
    delete e3;
    Event e4("Empty Event", 0, 5);
    return 0;
}
```

main.cpp | Clear | Run ▶

```
#include <iostream>
#include <string>
class Event
{
    int rows;
    int seatsPerRow;
    char** seats;
    std::string name;
public:
    Event(const std::string& eventName, int r, int s)
        : rows(r), seatsPerRow(s), seats(nullptr),
    name(eventName)
    {
        if (rows <= 0 || seatsPerRow <= 0)
```

Output

```
Event created: Stack Event
Event created: Block Event
O O O
O X O
O O O
Event destroyed: Block Event (3 x 3)
Event created: Heap Event
Event destroyed: Heap Event (2 x 3)
Event created with empty seat map
Event destroyed: Empty Event (0 x 0)
Event destroyed: Stack Event (2 x 4)
```

Memory Layout

```
Event
|
+-- seats
    |
    +--> row 0: [O][O][O][O]
    |
    +--> row 1: [O][O][O][O]
    |
    +--> row 2: [O][O][O][O]
```

seats is an array of row pointers.

Each row is separately allocated with new[].

Constructor / Destructor Order

- Stack objects are destroyed automatically in reverse construction order when their scope ends.
- An object inside a block is destroyed when that block ends.
- A heap object created by new is destroyed only when delete is called.
- For the heap object, delete invokes the destructor and then releases the object memory.
- delete[] must be used for arrays allocated with new[]. Using delete instead of delete[] is undefined behaviour.
- A destructor takes no arguments, has no return type, and cannot be overloaded because destruction is automatically selected for the object.

Memory Requirement

For a rows × seatsPerRow char map, the actual seat characters use rows × seatsPerRow bytes, plus the row-pointer array and allocation overhead. The tested sizes are 10×20 = 200 seats, 100×200 = 20,000 seats and 1,000×2,000 = 2,000,000 seats. Record the actual process/allocation measurements from the sanitizer or system when submitting.

RAII Conclusion

RAII ties resource release to object lifetime. When the Event leaves its scope, its destructor automatically releases the seat map. This greatly reduces the chance of forgetting cleanup and leaking memory.

Predict the Logger Output

For the Logger program, the expected logical sequence is:

```
+A
+B
-B
copyAc
use
-Ac
+P
-P
```

```
end
-A
```

The exact formatting depends on the spaces printed by the supplied program. The important order is: A is constructed; B is constructed and destroyed at block exit; passing a by value creates a copy Ac; that copy is destroyed after use; P is constructed and destroyed by delete; end is printed; finally A is destroyed.

Q4 – Overloading Arithmetic and Stream Operators

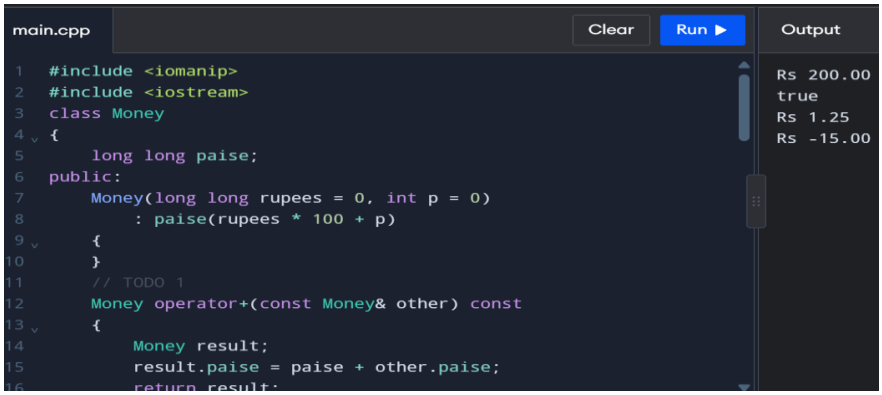
Money stores the total amount as paise in a long long. This avoids floating-point rounding problems for ordinary rupee/paise arithmetic.

Code

```
#include <iomanip>
#include <iostream>
class Money
{
   long long paise;
public:
   Money(long long rupees = 0, int p = 0)
      : paise(rupees * 100 + p)
   {
   }
   // TODO 1
   Money operator+(const Money& other) const
   {
      Money result;
      result.paise = paise + other.paise;
      return result;
   }
   Money operator-(const Money& other) const
   {
      Money result;
      result.paise = paise - other.paise;
      return result;
   }
   Money& operator+=(const Money& other)
   {
      paise += other.paise;
      return *this;
   }
   // TODO 2
   bool operator==(const Money& other) const
   {
      return paise == other.paise;
   }
   bool operator<(const Money& other) const
```

```
    {
        return paise < other.paise;
    }
    friend std::ostream& operator<<(std::ostream& out,
                                    const Money& m)
    {
        long long absPaise = (m.paise < 0) ? -m.paise : m.paise;
        out << "Rs " << (m.paise < 0 ? "-" : "")
            << absPaise / 100 << '.'
            << std::setw(2) << std::setfill('0')
            << absPaise % 100
            << std::setfill(' ');
        return out;
    }
    friend std::istream& operator>>(std::istream& in,
                                    Money& m)
    {
        long long rupees;
        int p;
        in >> rupees >> p;
        m.paise = rupees * 100 + p;
        return in;
    }
};
int main()
{
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n'
              << std::boolalpha
              << (c == Money(200)) << '\n';
    Money x(0, 75);
    Money y(0, 50);
    std::cout << x + y << '\n';
    Money negative = Money(10) - Money(25);
    std::cout << negative << '\n';
    return 0;
}
```

Expected output:



Design Answers

- + returns a new object because the original operands should remain unchanged. += modifies the left operand and therefore returns Money& so chained expressions such as a += b += c are possible.
- Arithmetic and comparison operators can be members because the left operand is a Money object.
- << must be a non-member because the left operand is std::ostream, not Money. It can be declared friend to access private paise.
- For a + b, the compiler conceptually calls a.operator+(b) when + is implemented as a member.
- All these operations take O(1) time and O(1) extra space.

Engineering Decision

Multiplying one Money amount by a scalar such as 3.0 can make sense. Multiplying Money by another Money generally does not represent an ordinary monetary operation, so it should not be provided without a specific domain meaning. Addition, subtraction, comparison and scalar multiplication are natural operations.

Q5 – Overloading Increment, Comparison and Subscript Operators

The Time class supports prefix/postfix increment and comparisons. Playlist overloads operator[] with both non-const and const versions and performs a range check.

Code

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time
{
   int hour;
   int minute;
   int second;
   void normalize()
   {
      if (second >= 60)
```

```
        {
            minute += second / 60;
            second %= 60;
        }
        if (minute >= 60)
        {
            hour += minute / 60;
            minute %= 60;
        }
    }
public:
    Time(int h = 0, int m = 0, int s = 0)
        : hour(h), minute(m), second(s)
    {
        normalize();
    }
    // Prefix ++
    Time& operator++()
    {
        ++second;
        normalize();
        return *this;
    }
    // Postfix ++
    Time operator++(int)
    {
        Time old = *this;
        ++second;
        normalize();
        return old;
    }
    bool operator==(const Time& other) const
    {
        return hour == other.hour &&
               minute == other.minute &&
               second == other.second;
    }
    bool operator<(const Time& other) const
    {
        if (hour != other.hour) return hour < other.hour;
        if (minute != other.minute) return minute < other.minute;
        return second < other.second;
    }
    friend std::ostream& operator<<(std::ostream& out,
                                    const Time& t)
    {
        out << t.hour << ':'
```

```
            << (t.minute < 10 ? "0" : "") << t.minute << ':'
            << (t.second < 10 ? "0" : "") << t.second;
        return out;
    }
};
class Playlist
{
    std::vector<std::string> tracks;
public:
    Playlist(std::initializer_list<std::string> list)
        : tracks(list)
    {
    }
    std::string& operator[](std::size_t i)
    {
        if (i >= tracks.size())
            throw std::out_of_range("Playlist index out of range");
        return tracks[i];
    }
    const std::string& operator[](std::size_t i) const
    {
        if (i >= tracks.size())
            throw std::out_of_range("Playlist index out of range");
        return tracks[i];
    }
    std::size_t size() const
    {
        return tracks.size();
    }
};
int main()
{
    Time t(0, 0, 59);
    Time a = t++;
    Time b = ++t;
    std::cout << "a = " << a << '\n';
    std::cout << "b = " << b << '\n';
    std::cout << "t = " << t << '\n';
    Time x(1, 0, 0);
    std::cout << std::boolalpha
              << (b == x) << '\n';
    Playlist p{"Song A", "Song B", "Song C"};
    std::cout << p[1] << '\n';
    try
    {
        std::cout << p[5] << '\n';
    }
```

```
    catch (const std::out_of_range& e)
    {
        std::cout << e.what() << '\n';
    }
    return 0;
}
```

Expected output:



Design Answers

- The compiler distinguishes postfix from prefix by the dummy int parameter in operator++(int). The integer is not used as a value; it only selects postfix syntax.
- Postfix returns the old value, so it needs a copy of the object. Prefix changes the object and returns *this by reference. Therefore prefix is generally preferable for class types when the old value is not needed.
- operator[] needs a non-const version to allow modification and a const version to allow read-only access through a const Playlist.
- An invalid index should throw std::out_of_range or use another clearly documented error policy. This solution uses checked access.
- Prefix and postfix increment are O(1). Playlist access is O(1), with the checked version performing one comparison before access.

Predicted Counter Output

```
Counter c{5};
Counter a = c++;   // a = 5, c = 6
Counter b = ++c;   // c = 7, b = 7

Output:
5 7 7
```

Postfix makes a copy of the old object. Prefix returns the modified object by reference.

Q6 – Rules of Operator Overloading

The Text class owns a dynamically allocated character array. The Rule of Three is used so copies, assignments and destruction manage the memory safely.

Code

```
#include <cstring>
#include <iostream>
class Text
{
   char* data;
public:
   Text(const char* s = "")
   {
      std::size_t n = std::strlen(s);
      data = new char[n + 1];
      std::strcpy(data, s);
   }
   Text(const Text& other)
   {
      std::size_t n = std::strlen(other.data);
      data = new char[n + 1];
      std::strcpy(data, other.data);
   }
   Text& operator=(const Text& other)
   {
      if (this != &other)
      {
         char* newData =
            new char[std::strlen(other.data) + 1];
         std::strcpy(newData, other.data);
         delete[] data;
         data = newData;
      }
      return *this;
   }
   ~Text()
   {
      delete[] data;
   }
   Text operator+(const Text& other) const
   {
      std::size_t n1 = std::strlen(data);
      std::size_t n2 = std::strlen(other.data);
      Text result;
      delete[] result.data;
      result.data = new char[n1 + n2 + 1];
      std::strcpy(result.data, data);
      std::strcat(result.data, other.data);
```

```
        return result;
    }
    bool operator==(const Text& other) const
    {
        return std::strcmp(data, other.data) == 0;
    }
    bool operator!=(const Text& other) const
    {
        return !(*this == other);
    }
    friend std::ostream& operator<<(std::ostream& out,
                                    const Text& t)
    {
        out << t.data;
        return out;
    }
};
int main()
{
    Text a("Hello ");
    Text b("World");
    Text c("!");
    Text result = a + b + c;
    std::cout << result << '\n';
    std::cout << std::boolalpha
              << (result == Text("Hello World!")) << '\n';
    Text empty;
    std::cout << "Empty: [" << empty << "]\n";
    Text copy = result;
    copy = copy;  // self-assignment
    std::cout << copy << '\n';
    return 0;
}
```

Expected output:

main.cpp Clear Run ▶ Output

```cpp
#include <cstring>
#include <iostream>
class Text
{
    char* data;
public:
    Text(const char* s = "")
    {
        std::size_t n = std::strlen(s);
        data = new char[n + 1];
        std::strcpy(data, s);
    }
    Text(const Text& other)
```

```
Hello World!
true
Empty: []
Hello World!
```

Rules of Operator Overloading

- Operators that cannot be overloaded include ::, ., .*, ?: and sizeof.
- Operator precedence and the number of operands (arity) cannot be changed.
- At least one operand of an overloaded operator must be a user-defined type.
- operator= must be a member function.
- A friend function is useful when the left operand is not the class type or when symmetric access is desired. Stream insertion << is the standard example because std::ostream is the left operand.
- Self-assignment is prevented by checking this != &other.

Complexity of a + b + c

If the three strings have lengths n1, n2 and n3, concatenating a+b copies O(n1+n2) characters into a temporary. Appending c then copies O(n1+n2+n3) characters. Overall character-copy work is O(n1+n2+n3). The intermediate temporary requires additional memory proportional to the intermediate string length.

std::string is normally preferred in real programs because it already provides safe ownership, copying, concatenation and memory management. A custom Text class is useful for learning resource management, implementing a specialized storage format, or demonstrating the Rule of Three/Five.

Q7 – Design a Complex-Number Class from First Principles

The Complex class represents AC impedance as a real part and an imaginary part. Resistors are R+0j, inductors are 0+jωL and capacitors are 0-j/(ωC).

Stage 1 – Interpret the Problem

- Data: two double values, real and imaginary.
- Operations: addition, subtraction, multiplication, division, equality and stream output.
- Constructors: default, real-only and real-plus-imaginary.
- Requirements: every arithmetic operation must return the mathematically correct complex value; division by zero must be detected; output must show rectangular form and magnitude.

Stage 2 – Complete Class

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>
class Complex
{
   double re;
   double im;
public:
   Complex() : re(0.0), im(0.0) {}
   Complex(double real) : re(real), im(0.0) {}
   Complex(double real, double imag)
      : re(real), im(imag)
   {
   }
   double real() const
   {
      return re;
   }
   double imag() const
   {
      return im;
   }
   double magnitude() const
   {
      return std::sqrt(re * re + im * im);
   }
   Complex operator+(const Complex& z) const
   {
      return Complex(re + z.re, im + z.im);
   }
   Complex operator-(const Complex& z) const
   {
      return Complex(re - z.re, im - z.im);
   }
   Complex operator*(const Complex& z) const
   {
      return Complex(
         re * z.re - im * z.im,
         re * z.im + im * z.re
      );
   }
   Complex operator/(const Complex& z) const
   {
      double denom = z.re * z.re + z.im * z.im;
      if (denom == 0.0)
```

```
            throw std::runtime_error(
                "Division by zero complex number");
        return Complex(
            (re * z.re + im * z.im) / denom,
            (im * z.re - re * z.im) / denom
        );
    }
    bool operator==(const Complex& z) const
    {
        const double EPS = 1e-9;
        return std::fabs(re - z.re) < EPS &&
               std::fabs(im - z.im) < EPS;
    }
    friend std::ostream& operator<<(std::ostream& out,
                                    const Complex& z)
    {
        out << std::fixed << std::setprecision(4)
            << z.re
            << (z.im >= 0 ? " + " : " - ")
            << std::fabs(z.im) << 'j';
        return out;
    }
};
int main()
{
    const double pi = 3.14159265358979323846;
    double frequency = 50.0;
    double w = 2.0 * pi * frequency;
    double R = 10.0;
    double L = 0.01;
    double C = 0.0001;
    Complex Zr(R, 0.0);
    Complex Zl(0.0, w * L);
    Complex Zc(0.0, -1.0 / (w * C));
    Complex series = Zr + Zl + Zc;
    std::cout << "R = " << Zr << '\n';
    std::cout << "L = " << Zl << '\n';
    std::cout << "C = " << Zc << '\n';
    std::cout << "Series = " << series << '\n';
    std::cout << "Series magnitude = "
              << series.magnitude() << '\n';
    try
    {
        Complex parallel =
            (Zr * Zl) / (Zr + Zl);
        std::cout << "Parallel = "
                  << parallel << '\n';
```

```
    std::cout << "Parallel magnitude = "
              << parallel.magnitude() << '\n';
    Complex zero;
    Complex bad = Zr / zero;
    std::cout << bad << '\n';
  }
  catch (const std::exception& e)
  {
    std::cout << "Error: " << e.what() << '\n';
  }
  return 0;
}
```



Formula Used

(a + bj) + (c + dj) = (a+c) + (b+d)j

(a + bj) - (c + dj) = (a-c) + (b-d)j

(a + bj)(c + dj)
= (ac-bd) + (ad+bc)j

(a + bj)/(c + dj)
= [(ac+bd) + (bc-ad)j] / (c^2+d^2)

|a+bj| = sqrt(a^2+b^2)

Series and Parallel RLC

For series impedance, add the resistor, inductor and capacitor impedances. For parallel impedance, use (Z1 × Z2)/(Z1 + Z2) for the selected branches. The denominator must be checked before division. The program above demonstrates the required safe failure for division by a zero complex number.

Stage 4 – Defend the Design

- std::complex<double> already provides a tested complex-number implementation and is preferred in production code.
- A custom class is useful when the application needs special formatting, validation, logging or domain-specific rules.
- Each constructor provides a convenient way to create the three required forms of complex numbers.

- Each arithmetic operator is O(1) and uses O(1) extra space.
- One limitation is exact floating-point equality; a tolerance is used because floating-point arithmetic can introduce tiny rounding errors.

Compared with std::complex<double>, the custom class has essentially the same asymptotic time and storage: two doubles and constant-time arithmetic. std::complex is generally safer and more thoroughly tested, while the custom class gives complete control over validation and presentation.

Experimental Evidence to Fill After Running

The assignment asks for predictions, compiler warnings, sanitizer reports, memory measurements and timing measurements. These values depend on the student's compiler, machine and optimization settings, so they should be captured from the actual execution rather than invented.

Question	Experiment	Result to Record	Status
Q1	Negative copies; -Wreorder; remove default constructor	Compiler output / warning	Fill from run
Q2	ASan shallow vs deep; self-assignment; copy counts	Sanitizer report and timings	Fill from run
Q3	ASan leak; 10×20, 100×200, 1000×2000	Memory/leak report	Fill from run
Q4	1,000,000 additions; negative subtraction	Measured time and result	Fill from run
Q5	10,000,000 ++ at -O0/-O2; invalid indexes	Measured times and exceptions	Fill from run
Q6	ASan; 100,000 concatenations; self-assignment	Sanitizer and timing	Fill from run
Q7	Series/parallel calculation; zero division	Calculated values and error	Completed by code